

Baggage_id Details							Tracking				
	passenger_id	flight_number	airline	departure_airport	destination_airport	check_in_time	current location	last_updated_time	status	remarks	transit_airport
1001	77987987	EK77	XYZ AIRLINE	JKF	MAA	XX:XX:XX	DXB				

Figure 6: Grouped data

immutable way of referring to entities. Row keys are always unique and act as the primary index in Apache HBase. Secondary indexes are not supported in HBase and can be created by periodically updating a secondary table or using the co-processor framework. In Figure 6, Baggage_id is the row key, Details and Tracking are column families, while passenger_id, flight_number, etc, are all columns.

Columns are addressed as a combination of the column family name and the column qualifier is expressed as family:qualifier. All members of a column family have the same prefix. In the preceding example, the passenger_id and flight_number column qualifiers can be referenced as Details: passenger_id and Details: flight_number, etc.

In Apache HBase, all rows are always sorted lexicographically by row key. The sort is byte-ordered and each row key is compared at a binary level, byte by byte, from left to right.

Open table format

Table formats organise data files aiming to incorporate database-like functionalities into data lakes. Apache Hive is a pioneering and widely adopted table format. However, it has certain drawbacks, including:

- Outdated table statistics related to partition life cycle.
- The requirement for users to possess knowledge about the physical data layout.
- Absence of lineage and historical tracking.
- Lack of support for schema evolution.
- Inefficient for small updates; rewrites entire partitions for even very small updates.
- Lacks ACID compliance.
- Has row level upserts.

These drawbacks can be overcome

by using open source Apache Iceberg combined with HBase and Apache Spark, which are all available out-of-the-box on AWS EMR.

The five key features of Apache Iceberg that help to combine with HBase, RDS and Spark queries are expressive SQL queries, hidden partitioning, schema evolution, time travel, and data compaction. These enable a comprehensive and efficient data management solution for data lake tables. They address common challenges in data lake environments and perform complex operations while maintaining data integrity and query performance.

We can use JDBC catalog, HBase catalog, S3 catalog and Spark catalog, and leverage Iceberg's catalogue switching facility.

The solution

Efficient baggage management is a critical aspect of the aviation industry. As the baggage journey involves multiple airlines and airports, it necessitates robust data management systems that can handle vast amounts of data while ensuring accuracy, reliability, and real-time access. Let us create an innovative airline baggage management system capable of handling the baggage journey life cycle. We will follow the IATA provided BIX messaging standard for our baggage information and streamlining operations while providing real-time insights.

Our modern BMS architecture will centre around data lakehouses, powered by a synergistic combination of AWS supported open source services like EMR HBase, RDS PostgreSQL, S3, EMR Apache Spark, Apache Iceberg, along with the usage of its catalogs (JDBC, Spark, and HBase).

Lakehouse architecture: The lakehouse architecture combines the best aspects of data warehouses and

data lakes, enabling efficient storage, processing, and analysis of data. It leverages the strengths of S3 as a scalable and durable storage system while incorporating features such as data versioning, schema evolution, and ACID-compliant transactions.

Apache Iceberg is integrated into the lakehouse architecture to provide data versioning, schema evolution, and table management capabilities. It ensures consistent and reliable snapshots of the 'baggage_tracking' table in the EMR HBase, facilitating analysis and enabling rollback to previous data versions if required.

S3 serves as a reliable and scalable storage system within the lakehouse architecture. It acts as a central data repository, enabling seamless integration with other systems involved in the baggage journey life cycle. S3 allows archival storage of raw BIX data, backups, and large media files associated with baggage events.

Data ingestion: Baggage data, following the IATA BIX format, is collected from various sources such as check-in systems, scanning devices, and baggage handling systems. The data is ingested into the system and stored in an S3 bucket as raw files, maintaining the original BIX format. Each bag event or BIX message is stored as a separate file with a unique identifier.

RDS PostgreSQL - metadata and passenger information: Passenger information, including flight details, passenger names, and baggage configuration, are stored in RDS PostgreSQL. The 'passenger_info' table serves as a centralised repository for passenger-related data. It updates the baggage tracker table on the open table in real-time using Iceberg's JDBC catalog.

EMR HBase - baggage events and tracking: Baggage tracking information extracted from the BIX data, such as bag